ECS7026P - Neural Networks and Deep Learning - 2025/26

Group - H:
Marharyta Dzhafarova 240016742, Saif Ali Khan Mangan 250977026, Haadi Muhammad Iftikhar 250582082

Task 1 - CIFAR-10 image classification

1. 1 Model Architecture

For Part 1, we implemented a convolutional neural network for CIFAR-10 image classification based on the required intermediate block architecture in Section 1.2. The model uses five Intermediate Blocks followed by an Output Block, with channel depth increasing progressively from 3 to 64, 128, 256, and 512. MaxPool2d layers are inserted after Block 2, Block 3, and Block 5 to reduce spatial dimensions from 32×32 down to 4×4.

1.2 Intermediate Block Design

Each block contained three independent convolutional branches (1×1, 3×3, 5×5) with padding set to preserve spatial dimensions. The input was globally average pooled channel-wise and passed through a fully connected layer to generate three scalar coefficients, which were used to weight and sum the branch outputs. Batch normalisation and GELU activation were applied after the weighted sum, Removing these additions reduces the block to the basic architecture described in Section 1.2.

1.3 Training and Hyperparameters

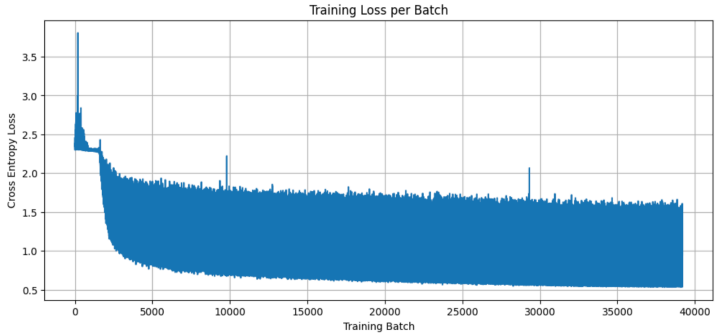
The model was trained for 200 epochs with a batch size of 256 using SGD with momentum 0.9 and weight decay 5e-4. The initial learning rate was set to 0.05 with a cosine annealing schedule. Cross-entropy loss with label smoothing of 0.1 was used, along with gradient clipping at norm 1.0. A random seed of 42 was set for reproducibility.

1.4 Additions Beyond the Basic Architecture (section 1.2)

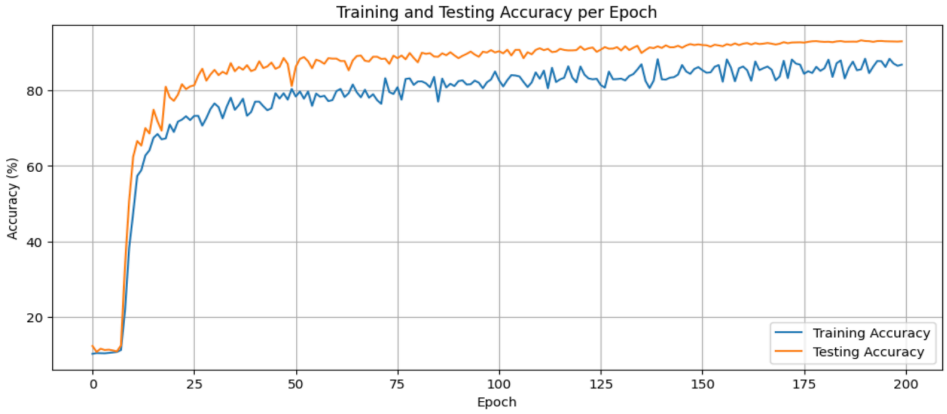
Four additions were made beyond the basic architecture. Batch normalisation was added after the weighted sum in each Intermediate Block to stabilise training and prevent internal covariate shift. A GELU activation was applied after Batch Normalisation for smoother gradient flow compared to ReLU. MaxPool2d layers were inserted between stages in the full model to reduce spatial dimensions and increase the receptive field. Finally, Dropout with p=0.2 was added in the Output Block to reduce overfitting.

1.5 Results

The best model achieved a test accuracy of **93.3%** at epoch **190**, exceeding the **≥92%** threshold in the marking criteria.



From the plot above, Loss dropped steadily over 200 epochs. Occasional spikes are visible, likely caused by CutMix batches with unusual mixing ratios, this is normal and does not affect overall training. The general downward trend confirms the model was learning consistently.



From the training and testing accuracy plot above, both training and testing accuracy improved steadily and did not show significant divergence, suggesting limited overfitting. Testing accuracy remained higher than training accuracy throughout, this is expected because CutMix and random erasing make training images harder than the clean test images. Testing accuracy stabilised around 92–93% after epoch 150, with the best result of 93.3% at epoch 190. The final model remained clearly based on the required intermediate block design, with standard regularisation and optimisation techniques added to improve generalisation.

Task 2 - Beyond CIFAR-10 image classification

2.1 Models and Dataset

Introduction

For this part of the assignment, we chose to implement a Named Entity Recognition (NER), which is a sequence labelling task where the goal is to assign a named entity tag to every token in an input sentence.

In other words, the task is predicting the label at each position y_t in the input $x_1, x_2, \ldots, x_T$. NER is also a well-established benchmark for comparing sequence modelling architectures. The biggest challenge with performing this task is that entity labels are context-dependent: the same word can belong to different entity types depending on what surrounds it. For example, "Washington" can be a person, a location, or part of an organisation's name depending on context. This means that the model cannot capture the label in isolation and needs to capture the structure of the sequence to perform well.

Dataset and Models

We chose WikiANN English to conduct the experiments with. It was constructed by automatically labelling Wikipedia text using existing knowledge bases, and covers three entity types. PER (person), ORG (organisation), and LOC (location), encoded in the standard BIO scheme, where B- marks the beginning of an entity, I- marks its continuation, and O marks non-entity tokens. The dataset contains 20,000 training sentences, 10,000 validation sentences, and 10,000 test sentences, with an average sentence length of approximately 8 tokens. WikiANN was chosen over the originally proposed CoNLL-2003 benchmark due to a HuggingFace API deprecation that prevented CoNLL-2003 from loading. Although WikiANN is considered a well-suited alternative for the experiments, CoNLL-2003 includes one more entity type, MISC.

We implemented and compared 2 architectures: Bidirectional LSTM (BiLSTM) and a Transformer encoder, both trained from scratch on the WikiANN training set. The BiLSTM processes the input sequence simultaneously left-to-right and right-to-left, concatenating the two hidden states at each position to produce a token representation informed by its full surrounding context. It was chosen as a baseline because of its built-in assumption that word order matters and nearby tokens are more relevant to each other, which is well matched to the relatively short sentences in WikiANN.

The Transformer uses self-attention to measure relationships between all token pairs simultaneously, regardless of their distance in the sequence. Since this operation has no built-in notion of order, sinusoidal positional encodings are added to the token embeddings before the attention layers to provide positional information. Without this, the model would treat every sentence as an unordered bag of tokens. The Transformer was chosen as the main model of interest because it represents the dominant paradigm in modern sequence modelling. Both architectures use the same input and output layers, so any difference in performance can be attributed to the architecture itself rather than other factors.

Hyperparameters were selected based on the computational constraints of the experiments. Embedding dimension 128, hidden dimension 128 (BiLSTM) or feed-forward dimension 256 (Transformer), 2 layers, 4 attention heads (Transformer only), dropout 0.3, batch size 32, Adam optimiser with learning rate 10^(-3), and gradient clipping at norm 1.0. Early stopping with patience 15 was applied, monitoring validation F1, with a maximum of 60 training epochs to allow the Transformer sufficient time to converge. All experiments were repeated across 3 random seeds (42, 123, 7) and results are reported as mean ± standard deviation. The BiLSTM has approximately 4.8M trainable parameters and the Transformer approximately 4.4M.

2.2 Experimental protocol

We conducted 4 experiments, each of them designed to answer a specific question about how the 2 architectures behave on NER.

Experiment 1. Baseline comparison All 3 models, BiLSTM, Transformer and the Transformer without positional encoding, were trained under identical conditions and evaluated on the held-out test set using entity-level F1, computed with the seqeval library. Entity-level F1 was chosen over token-level accuracy because the majority of tokens in NER datasets are labelled O (non-entity). This can make accuracy misleading as the model that predicts O for every token would score above 80% accuracy, which is useless. F1 on the entity-level requires all tokens of a multi-word entity to be labelled correctly for it to count as a correct prediction. Per-entity F1 scores for PER, ORG, and LOC were also recorded to identify which entity types each architecture handles best. This experiment was motivated by Vaswani et al. (2017), who introduced the Transformer and compared it against recurrent baselines, and Yan et al. (2019), who demonstrated that when both architectures are trained from scratch on limited data, recurrent models tend to perform better. They implemented a similar experiment comparing vanilla Transformers to BiLSTMS on NER.

Experiment 2. Sentence length analysis. Test sentences were grouped into three bins: short (fewer than 10 tokens), medium (10 to 25 tokens), and long (more than 25 tokens). Then, the entity-level F1 was computed for each of the bins. F1 was computed per seed for each bin and then averaged. This experiment was motivated by the vanishing gradient problem described by Bengio et al. (1994), which predicts that recurrent models should struggle on longer sequences where gradients must propagate across many steps. The expectation that Transformers handle long-range dependencies better through direct attention is from Vaswani et al. (2017) and was tested empirically by Tran et al. (2018).

Experiment 3. Positional encoding ablation. A Transformer trained without sinusoidal positional encoding was compared against the full Transformer, with all other hyperparameters held constant. When positional encoding is removed, the model can no longer distinguish token order, which allows the contribution of positional information to be clearly isolated. This is directly addressed in Vaswani et al. (2017), where they compare sinusoidal positional encoding against learned positional embeddings.

Experiment 4. Dropout sweep. Both models were trained at dropout rates of 0.0, 0.3, and 0.5. The dropout=0.3 results from Experiment 1 were reused for this condition, requiring only 12 additional training runs (2 models × 2 dropout values × 3 seeds). This experiment was directly motivated by Srivastava et al. (2014), who introduced dropout and showed that the optimal rate depends on model capacity relative to dataset size. Merity et al. (2017) highlighted the importance of dropout regularisation for recurrent architectures.

2.3 Analysis

Experiment 1: Figure 1 shows the BiLSTM substantially outperforming the Transformer (F1: 0.668 vs 0.407). This contradicts the initial hypothesis but is consistent with existing literature. Yan et al. (2019) showed that vanilla Transformers trained from scratch on NER tasks often underperform BiLSTMs. Unlike the BiLSTM, which naturally captures sequential word order, the Transformer must learn this structure from the training data itself, putting it at a disadvantage with limited supervision. Figure 2 also shows that ORG is the hardest entity type for all models. The BiLSTM achieved only 0.556 F1 on ORG compared to 0.747 on PER and 0.706 on LOC, likely because organisation names are far more varied and inconsistent than person or location names. The Transformer follows the same ordering but loses proportionally more on ORG (54% of BiLSTM performance, vs. 64% on PER and LOC), reinforcing that vanilla Transformers struggle most where pattern-learning needs the most supervision.

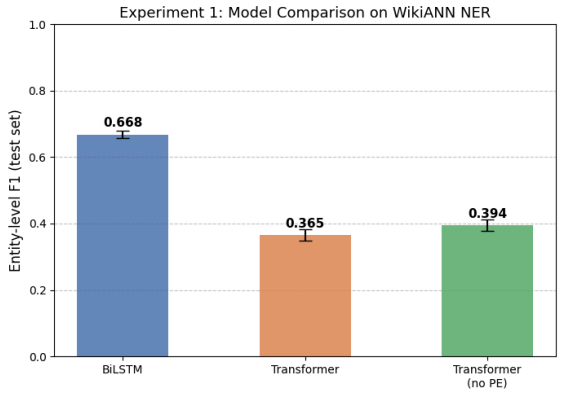


Figure 1

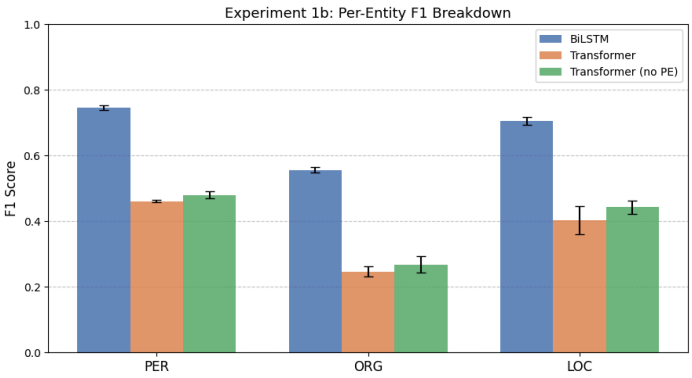


Figure 2

The learning curves (Figures 3 and 4) reveal an important detail: under the initial patience=5 setting, the Transformer had not converged — its validation F1 was still rising when training stopped. Patience was therefore increased to 15, and the Transformer retrained, after which the learning curves show clear convergence by epoch 35. Increasing patience from 5 to 15 improved Transformer F1 from 0.365 to 0.407.

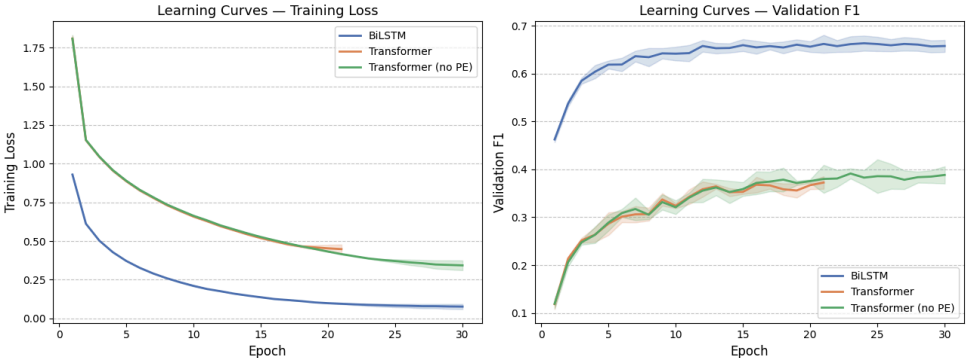


Figure 3 (patience = 5)

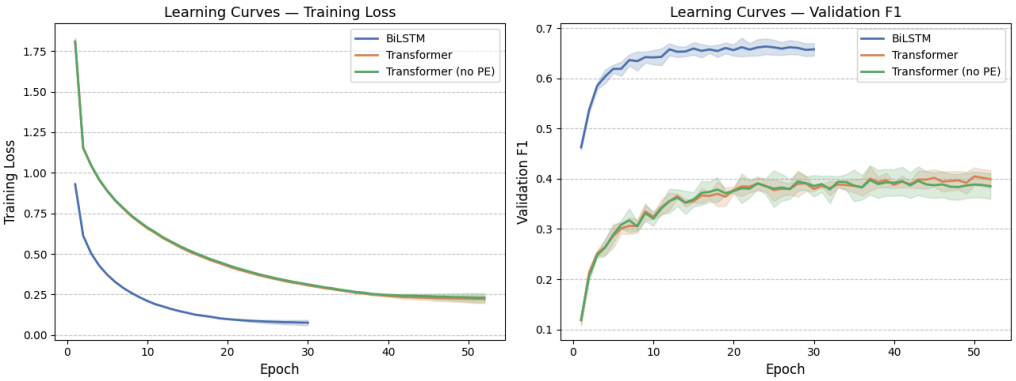


Figure 4 (patience = 15)

Experiment 2. Figure 5 shows the BiLSTM maintaining its lead across all length bins, contradicting the hypothesis that the Transformer would close the gap on longer sequences. Notably, both models perform best on the long-sentence bin (BiLSTM 0.741, Transformer 0.542), which likely reflects longer sentences containing more entities and richer disambiguating context rather than any architectural advantage.

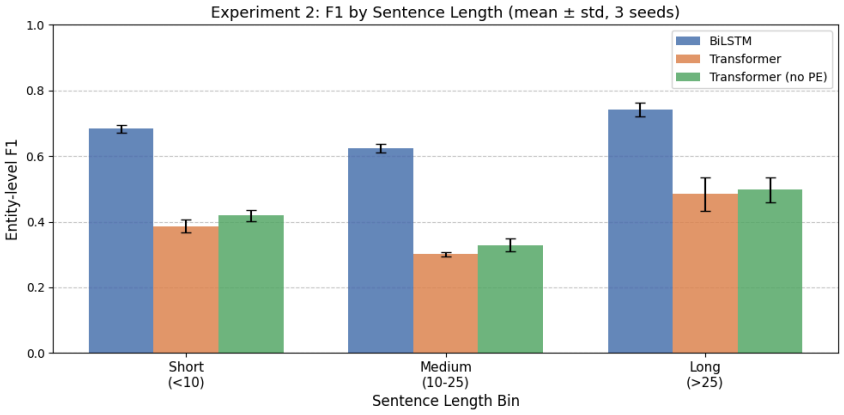


Figure 5

Experiments 3 and 4. Removing positional encoding causes a small F1 drop (0.407 to 0.398). While Vaswani et al. (2017) showed that PE is necessary for the Transformer to distinguish token order, the modest effect here is consistent with the short average sentence length in WikiANN. With most sentences being short, there is little word order ambiguity for the model to resolve, so it can partially compensate for the missing positional information. The BiLSTM is more sensitive in absolute terms to dropout removal (0.668 to 0.587) than the Transformer (0.365 to 0.285), and for both models, the best rate in the sweep was 0.5 rather than the 0.3 used in Experiment 1, which was fixed before the sweep was run.

Strengths and Limitations. The protocol has several strengths: results are averaged over three seeds with reported standard deviations, early stopping with patience=15 ensures fair convergence, and entity-level F1 is the standard metric for NER evaluation. The main weakness is that the dropout sweep used patience=5 rather than patience=15, meaning the Transformer values in Experiment 4 can be underestimated. WikiANN's automatically projected labels also introduce some noise to the experimentation, which would be absent with manually annotated labels.

References

Bengio, Y., Simard, P. and Frasconi, P. (1994) 'Learning long-term dependencies with gradient descent is difficult', IEEE Transactions on Neural Networks, 5(2), pp. 157–166.

Hendrycks, D. and Gimpel, K. (2016) 'Gaussian error linear units (GELUs)'. Available at: https://arxiv.org/abs/1606.08415.

Ioffe, S. and Szegedy, C. (2015) 'Batch normalization: Accelerating deep network training by reducing internal covariate shift', *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.

Krizhevsky, A. (2009) *Learning multiple layers of features from tiny images*. Technical Report. University of Toronto. https://www.cs.toronto.edu/~kriz/cifar.html

Loshchilov, I. and Hutter, F. (2017) 'SGDR: Stochastic gradient descent with warm restarts', *International Conference on Learning Representations (ICLR)*. Available at: https://arxiv.org/abs/1608.03983.

Merity, S., Keskar, N.S. and Socher, R. (2017) 'Regularizing and optimizing LSTM language models', arXiv preprint arXiv:1708.02182.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', Journal of Machine Learning Research, 15(1), pp. 1929–1958.

Tran, K., Bisazza, A. and Monz, C. (2018) 'The importance of being recurrent for modeling hierarchical structure', Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, pp. 4731–4736.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', Advances in Neural Information Processing Systems, 30, pp. 5998–6008.

Yan, H., Deng, B., Li, X. and Qiu, X. (2019) 'TENER: adapting transformer encoder for named entity recognition', arXiv preprint arXiv:1911.04474.

Yun, S., Han, D., Oh, S.J., Chun, S., Choe, J. and Yoo, Y. (2019) 'CutMix: Training strategy that makes use of sample pairs', *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*

Zhong, Z., Zheng, L., Kang, G., Li, S. and Yang, Y. (2020) 'Random erasing data augmentation', *Proceedings of the AAAI Conference on Artificial Intelligence*